

Reinforcement Learning

Unit 1

- Introduction: Elements of Reinforcement Learning, Limitations and Scope, An Extended Example: Tic-Tac-Toe.
- Tabular Solution Methods: An n-Armed Bandit Problem, Action-Value Methods, Incremental Implementation, Tracking a Nonstationary Problem, Optimistic Initial Values, Upper-Confidence Bound Action Selection, Gradient Bandits.

Machine learning: Definition

- Machine learning is a scientific discipline that is concerned with the design and development of algorithms that allow computers to learn based on data, such as from sensor data or databases.

A major focus of machine learning research is to automatically learn to recognize complex patterns and make intelligent decisions based on data .

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map $x \rightarrow y$

Examples: Classification,
regression, object detection,
semantic segmentation, image
captioning, etc.



→ Cat

Classification

Unsupervised Learning

Data: x

Just data, no labels!

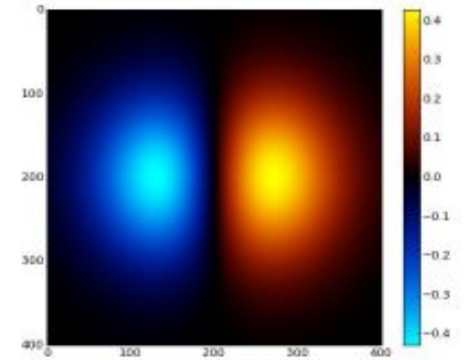
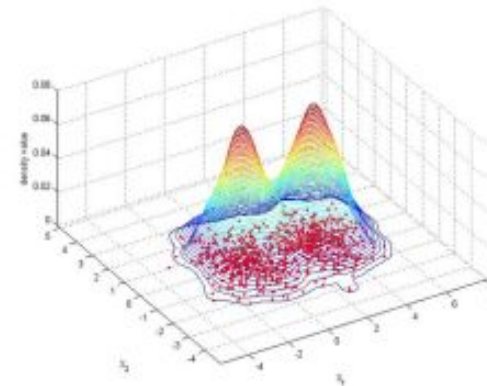
Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.



Figure copyright Ian Goodfellow, 2016. Reproduced with permission.

1-d density estimation

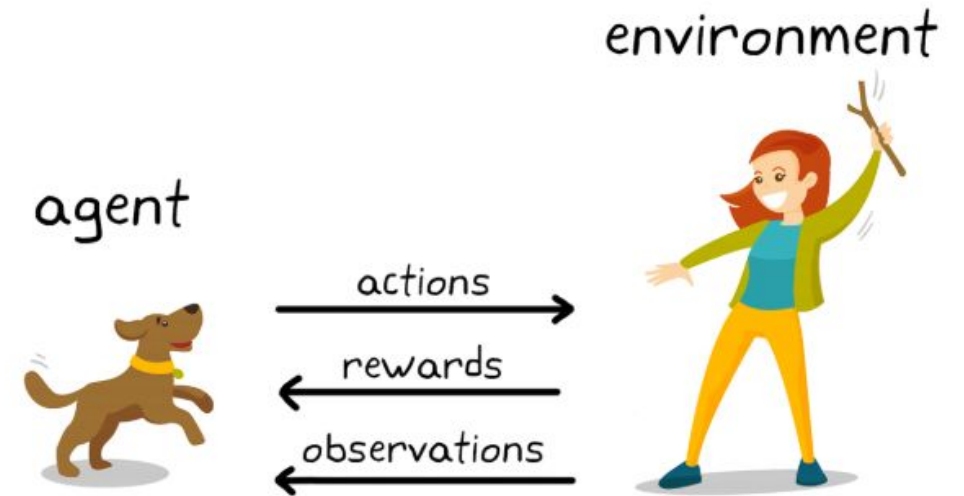


2-d density estimation

Reinforcement Learning

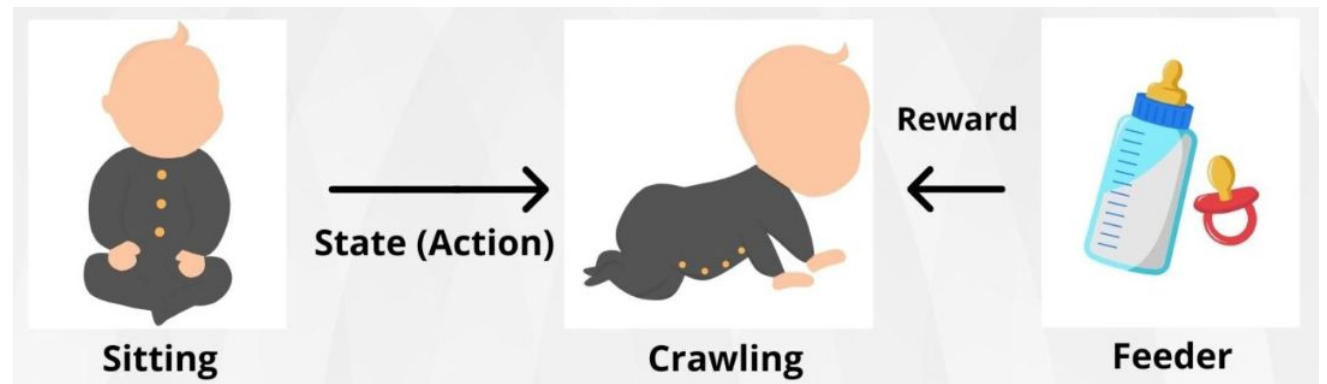
Reinforcement learning is a type of machine learning technique where a computer agent learns to perform a task through repeated **trial and error** interactions with a dynamic environment.

This learning approach enables the agent to make a series of decisions that maximize a **reward metric** for the task without human intervention and without being explicitly programmed to achieve the task.



Key Idea:

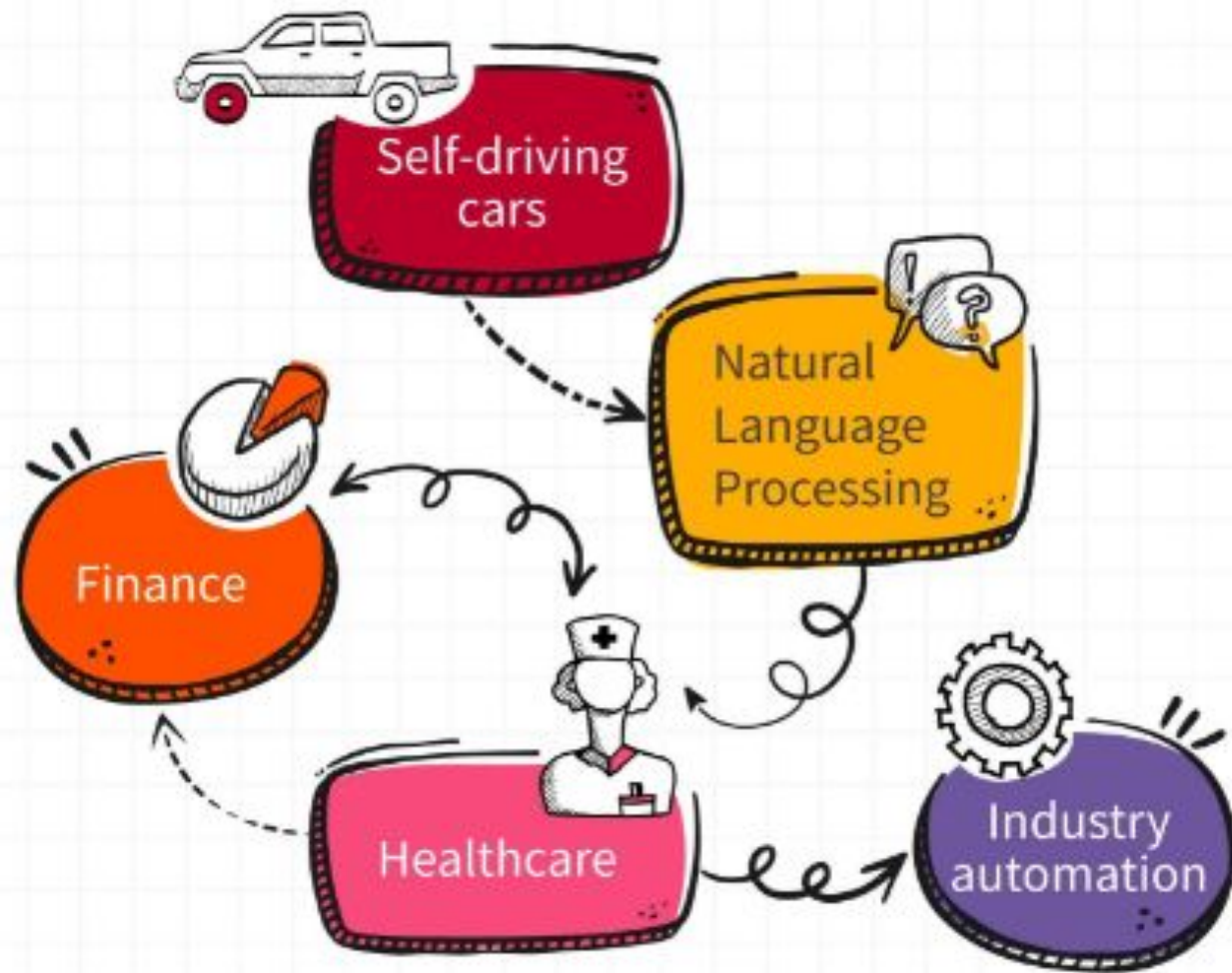
- Trial & error learning
- Rewards & penalties guide learning



Aspect	Supervised Learning	Unsupervised Learning	Reinforcement Learning (RL)
Goal	Learn a mapping from inputs to outputs using labeled data.	Discover hidden patterns or structures in unlabeled data.	Learn a policy to make decisions by interacting with an environment.
Data	Labeled data (input-output pairs).	Unlabeled data (only input features).	Data consists of actions, states, and rewards from the environment.
Learning Process	Model learns from examples with known outputs.	Model finds structure without knowing the output labels.	Agent learns from interactions with the environment via trial and error.
Feedback	Immediate feedback (based on known labels).	No feedback (patterns are inferred without specific outcomes).	Delayed feedback (based on rewards or penalties after actions).
Goal Type	Minimize prediction error or maximize accuracy.	Identify patterns like clusters or reduce dimensionality.	Maximize cumulative reward over time (long-term goals).
Environment Interaction	No interaction with an environment.	No interaction with an environment.	Continuous interaction with the environment to make decisions.
Examples	Image classification, spam detection, regression tasks.	Customer segmentation, anomaly detection, clustering.	Robotics, game AI, autonomous vehicles.

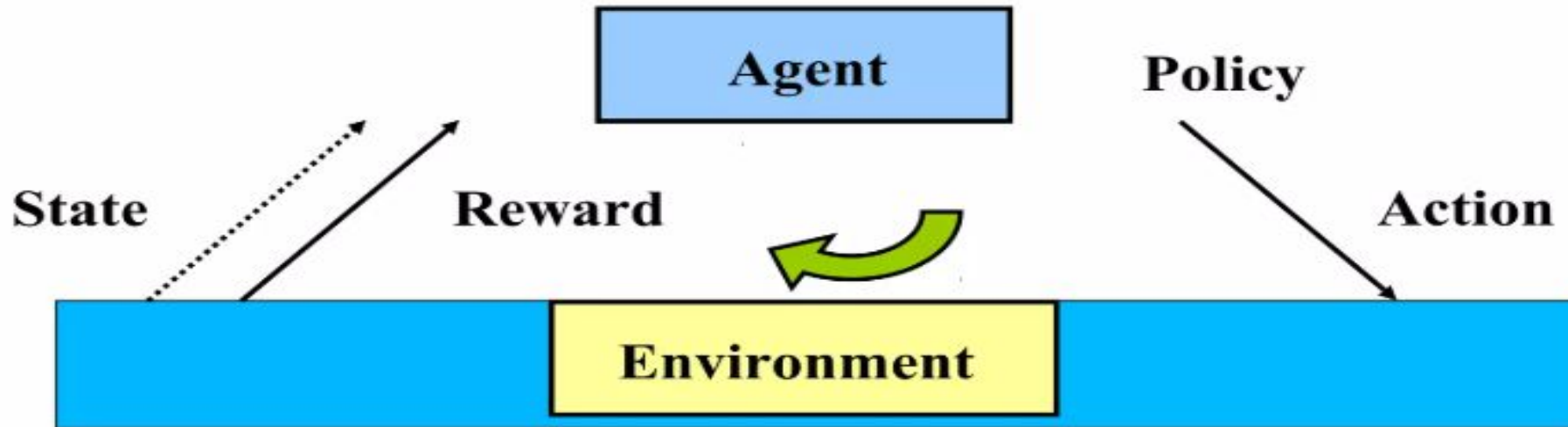
Supervised learns from answers, Unsupervised learns from patterns, RL learns from consequences.

Applications of reinforcement learning



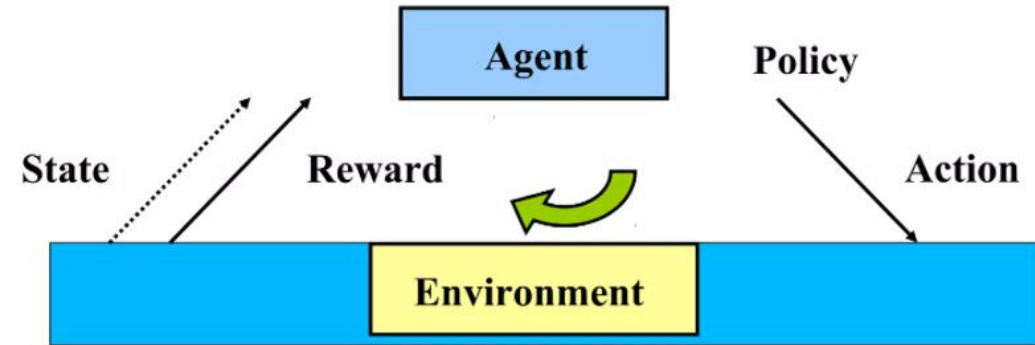
- 1. Robotics & Automation** 🤖 : Self-learning robots (Boston Dynamics, warehouse automation)
- 2. Game AI** 🎮 : AlphaGo, AlphaZero (beating human champions in Go, Chess)
- 3. Self-Driving Cars** 🚗 : Tesla Autopilot, Waymo self-driving systems
- 4. Finance & Stock Trading** 📈 : Algorithmic trading bots (predicting stock trends)
- 5. Healthcare & Drug Discovery** 🏥 : AI-based personalized treatment plans (e.g., chemotherapy optimization)
- 6. Smart Systems & IoT** ⚡ : Reinforcement learning for smart cities and automation
- 7. Recommendation Systems** 🎵📺 : Personalized content recommendations (Netflix, YouTube, Spotify)
- 8. Natural Language Processing (NLP) & Chatbots** 🤖💬 : AI assistants (ChatGPT, Alexa, Google Assistant)
- 9. Cybersecurity** 🔒 : Detecting cyber threats using RL-based anomaly detection
- 10. Industrial Process Optimization** 🏭 : Predictive maintenance for manufacturing equipment

Elements of Reinforcement Learning



- ❑ **Agent:** Intelligent programs (learner or decision maker)
- ❑ **Environment:** External condition
- ❑ **Policy:**
 - ❑ Defines the agent's behavior at a given time
 - ❑ A mapping from states to actions
 - ❑ Lookup tables or simple function

Policy (π)



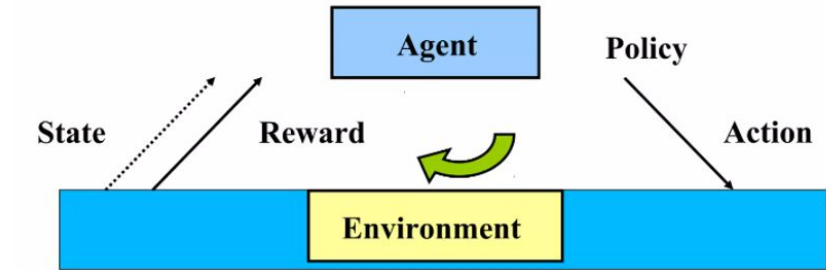
- A policy defines the learning agent's way of behaving at a given time. Roughly speaking, a policy is a mapping from perceived states of the environment to actions to be taken when in those states.
- In some cases the policy may be a simple function or lookup table, whereas in others it may involve extensive computation such as a search process.

A policy defines the agent's behavior—it maps states to actions.

$$\pi(s) \rightarrow a$$

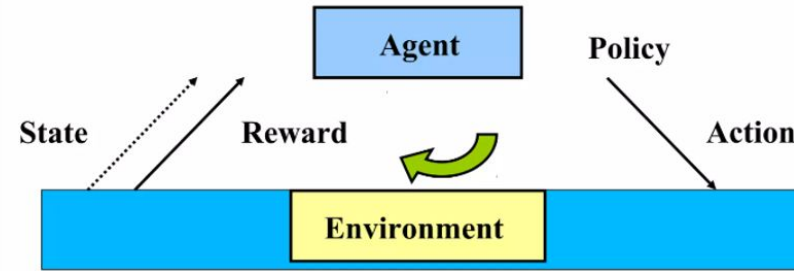
“When in this situation, do this action”

Rewards



- A reward signal defines the goal in a reinforcement learning problem.
- On each time step, the environment sends to the reinforcement learning agent a single number, a reward.
- The agent's sole objective is to maximize the total reward it receives over the long run.
- The reward signal thus defines what are the good and bad events for the agent.

Value Functions



- Whereas the reward signal indicates what is good in an immediate sense, a value function specifies what is good in the long run.
- The value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state.
- Rewards are in a sense primary, whereas values, as predictions of rewards, are secondary.

The value function estimates how good a state (or state–action pair) is in the long run.

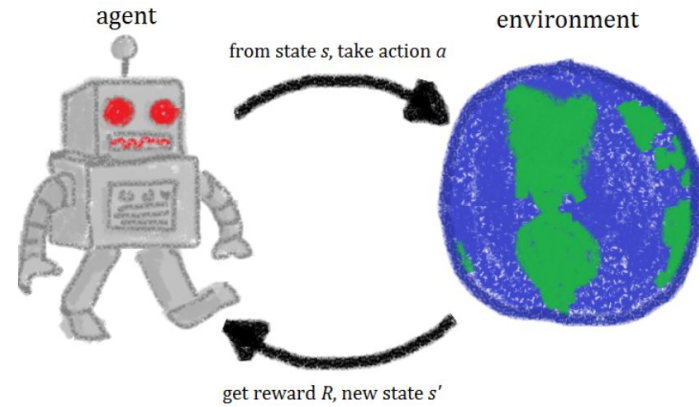
- **State-value:** $V(s)$
- **Action-value (Q-value):** $Q(s,a)$

Reward vs Value Function

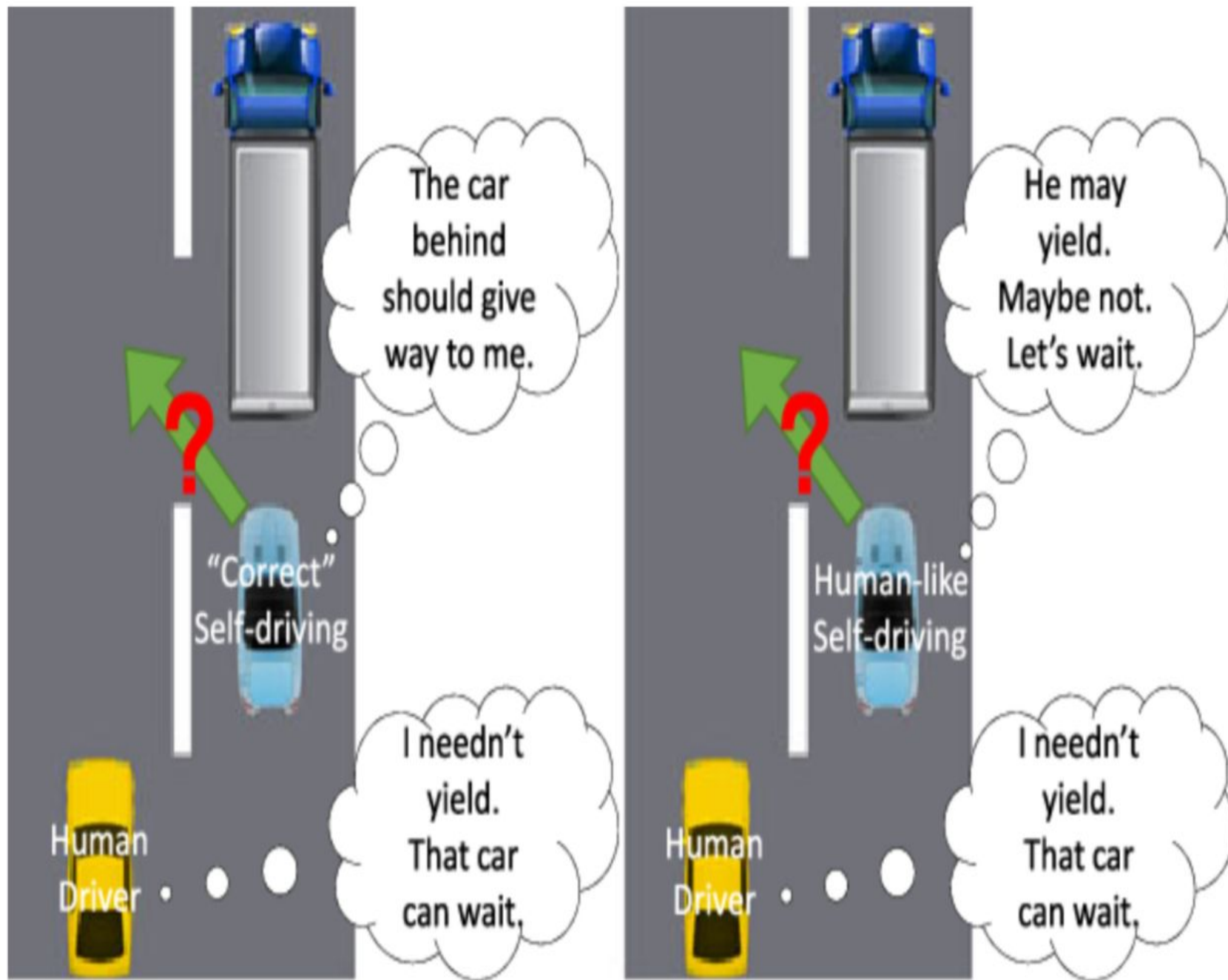
Aspect	Reward	Value Function
Meaning	Immediate feedback received after an action	Expected long-term benefit from a state or action
Time Perspective	Short-term (instant)	Long-term (future-oriented)
What it Answers	“How good was this action now ?”	“How good is this state/action in the future ?”
Nature	Direct signal from environment	Prediction made by the agent
Dependency	Depends only on current action and state	Depends on future rewards and policy
Role in Learning	Guides instant reaction	Guides planning and decision making
Changes With	Each step or action	Improves gradually through learning
Example (Student Life)	Got marks for today’s quiz	Overall CGPA after 4 years
Example (Food)	Taste of chocolate now 	Health impact over time 
Mathematical Form	(R_t)	($V(s)$), ($Q(s,a)$)
Primary or Secondary	Primary (given by environment)	Secondary (estimated from rewards)

Reward provides immediate feedback, while the value function estimates the long-term desirability of states or actions based on future rewards.

Models



- Model is something that mimics the behavior of the environment, or more generally, that allows inferences to be made about how the environment will behave.
- Models are used for planning, by which we mean any way of deciding on a course of action by considering possible future situations before they are actually experienced.
- Methods for solving reinforcement learning problems that use models and planning are called model-based methods.



RL Element	Real Life Meaning
Agent	The car 🚗
Environment	Roads, traffic, signals
Action	Accelerate, brake, turn
Reward	Safe ride (+10), accident (-100)
Policy	Driving style
Value Function	"Is this road safe long-term?"

The Reinforcement Learning Loop

1. Agent observes **state**
2. Agent selects **action**
3. Environment responds with:
 - **reward**
 - **next state**
4. Agent updates its policy
5. Process repeats

This loop continues until the agent learns an **optimal policy**.

1.Reinforcement Learning (RL) follows a unique approach compared to Supervised and Unsupervised Learning.

- i. Briefly explain the main goal of each learning type: Supervised, Unsupervised, and Reinforcement Learning.
- ii. Identify the key differences that set RL apart from other two types of Machine Learning.

Imagine a self-driving car that is learning to drive safely and efficiently in a busy city. The car must make decisions such as stopping at red lights, avoiding obstacles, and choosing the best route to its destination. Identify the elements of Reinforcement learning such as Agent, Environment, Policy, Reward and Value function for the given scenario.

Limitations & Scope

- RL methods are primarily focused on estimating value functions.
- Alternative methods like evolutionary algorithms can also solve RL problems without value functions.

Evolutionary Methods

- Methods such as genetic algorithms and simulated annealing evolve policies over time.
- These methods don't learn from individual actions but instead evaluate overall behavior.
- Effective in some cases, especially when state information is unclear.

Limitations of Evolutionary Methods

- Evolutionary methods do not use the structure of the RL problem (e.g., state-action relationships).
- Less efficient compared to RL methods that use detailed behavioral interactions.
- Do not learn during interaction with the environment unless evolving learning algorithms.

Reinforcement Learning is not the only way to solve sequential decision problems, but it is usually more efficient because it exploits state–action structure and learns continuously from interaction.

Aspect	Evolutionary Methods	Reinforcement Learning
Learning style	Generation-based	Step-by-step interaction
Feedback	Overall performance	Reward after each action
Uses state-action info	No	Yes
Efficiency	Lower	Higher
Adaptation speed	Slower	Faster
Analogy	Natural evolution	Human learning from experience

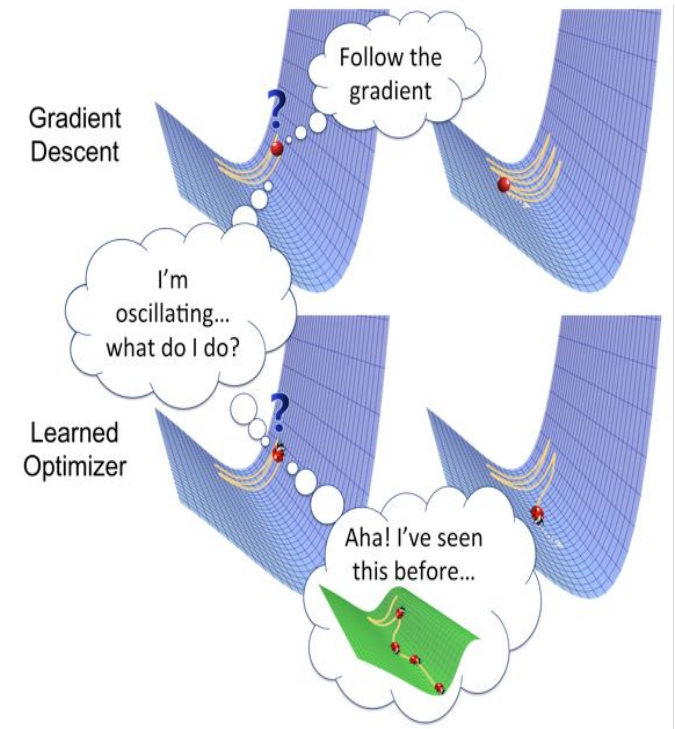
Evolution waits for generations, RL learns every second.

Policy Gradient Methods -

- Unlike evolutionary methods, policy gradient methods adjust policies during interaction with the environment.
- These methods estimate how to improve a policy by adjusting numerical parameters.
- More efficient because they use information from individual interactions to guide learning.

Reinforcement Learning and Optimization

- RL agents aim to maximize the reward, but achieving the maximum reward is not always possible.
- The focus is on improving the agent's reward over time, not necessarily achieving the maximum.



What “Policy Gradient” Means

- **Policy (π):** The agent's behavior — how it chooses actions in a given state
- **Gradient:** Direction in which parameters should be changed to improve performance

Policy Gradient methods directly learn and improve a policy by adjusting its parameters during interaction with the environment, using feedback from individual actions to efficiently maximize rewards.

Tic-Tac-Toe: RL vs Classical Methods

Why Reinforcement Learning is useful compared to classical methods.

RL in Tic-Tac-Toe

- The RL agent:
 - Plays many games
 - Observes wins, losses, and draws
 - Learns which moves lead to better outcomes
- If the opponent makes mistakes, RL:
 - **Notices patterns**
 - **Exploits weaknesses**
 - Improves its winning chances over time

Classical Methods – Why They Fall Short

Minimax

- Assumes the opponent always plays **optimally**
- Result:
 - Plays conservatively
 - Often settles for draws
 - Cannot exploit opponent mistakes

Dynamic Programming

- Requires:
 - Complete state transitions
 - Exact opponent behavior model
- In real games:
 - Opponent behavior is unknown
 - Model is unavailable or inaccurate

X	O	O
O	X	X
		X

“Reinforcement learning adapts to real opponents by learning from experience, whereas classical methods rely on idealized assumptions.”

Explain the advantages of Reinforcement Learning over classical methods using Tic-Tac-Toe as an example.

Consider Tic-Tac-Toe as an example to illustrate reinforcement learning and its advantages over classical approaches.

An RL-based player learns by playing multiple games and receiving feedback in the form of rewards for wins, losses, or draws. Over time, it adapts its strategy to exploit mistakes made by an imperfect opponent, thereby increasing its chances of winning.

In contrast, classical methods such as minimax assume an optimal opponent and therefore do not adapt to real gameplay, often resulting in conservative strategies. Dynamic programming approaches require a complete and accurate model of the opponent's behavior, which is usually unavailable in practice.

Reinforcement learning overcomes these limitations by learning directly from experience without requiring prior knowledge of the opponent, making it suitable for environments where opponent behavior is unknown or uncertain.

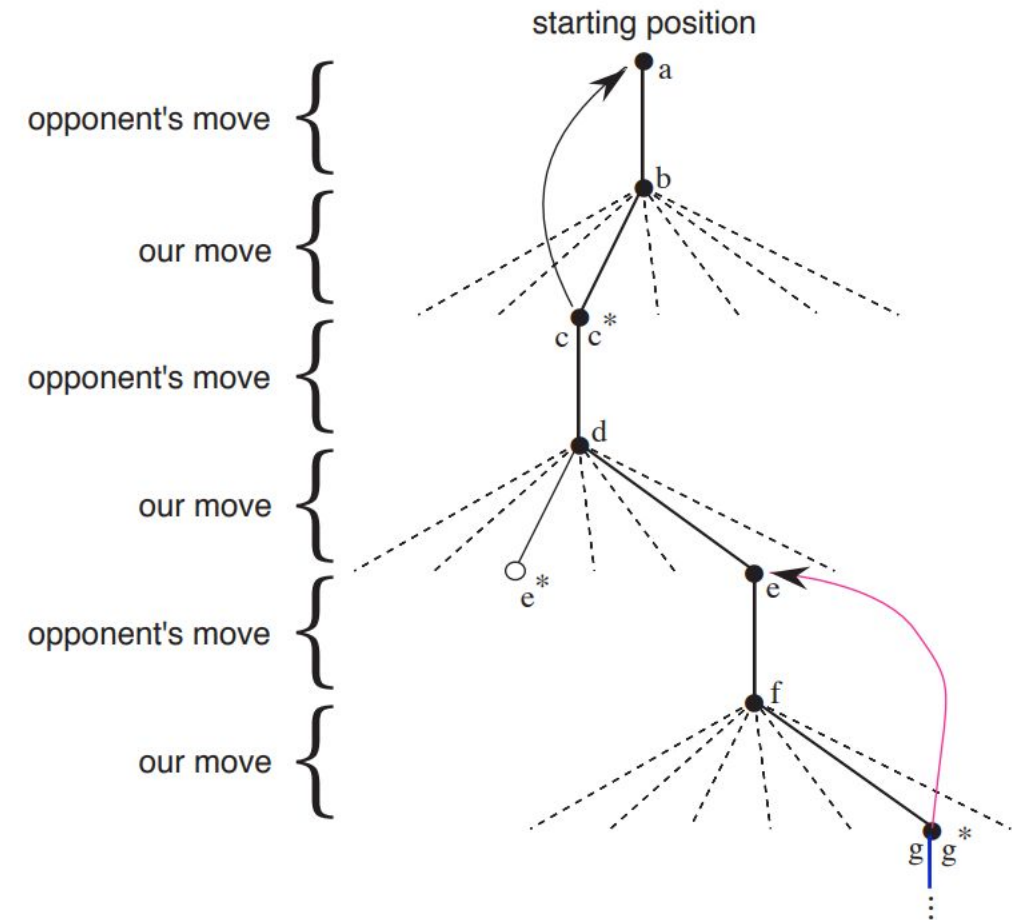
Reinforcement learning (RL) using a value function can help a player improve at **Tic-Tac-Toe** by learning from experience. Here's a breakdown in simple terms:

1. Using a Value Function

- We create a table where each possible game state gets a number (value) representing our estimated chance of winning.
- If a state has three **X**'s in a row, its value is **1** (since we won). If three **O**'s are in a row or the board is full, its value is **0** (since we lost or tied).
- All other states start with a guess of **0.5** (a 50% chance of winning).

2. Playing and Learning

- Each time we play, we look at possible moves and check their values.
- **Most of the time**, we choose the move with the highest value (best chance of winning).
- **Sometimes**, we pick a random move to explore new situations—this is called **exploration**.



3. Updating the Value Function (Learning from Mistakes and Successes)

- After every move, we update the values in our table.
- If a move leads to a good outcome, we increase the value of the previous state. If it leads to a bad outcome, we lower the value.
- This update happens step by step, gradually improving our estimates over many games.

4. Why This Works Well

- Over time, our values get more accurate, helping us play optimally against a given opponent.
- If our opponent changes their strategy, we can still adapt by keeping a small learning rate.

5. Comparison with Evolutionary Methods

- **Evolutionary methods** test entire strategies over many games before making any changes, without considering individual moves.
- **Value function methods** learn from each move in real-time, making them **faster and more efficient** at improving gameplay.

Analyse how learning occurs in the Tic-Tac-Toe example using value functions.

Numerical example of Tic-Tac-Toe using Reinforcement Learning (RL) with a Value Function.

Step 1: Initial Setup – Assigning Values to Game States

We create a table where each possible Tic-Tac-Toe board state has a value representing our **estimated chance of winning**.

- Winning states (e.g., **three Xs in a row**) get a value of **1**.
- Losing states (e.g., **three Os in a row**) get a value of **0**.
- All other states start at **0.5** (since we don't know yet if they are good or bad).

Board State	Value (V)
X X X _ _ _ _ _	1 (Win)
O O O _ _ _ _ _	0 (Loss)
_ X O _ _ _ _ _	0.5 (Unknown)
_ _ _ _ _ _ _ _	0.5 (Unknown)

Step 2: Playing a Game with Learning

We now play a game against an opponent and **update our values based on the result.**

Game Progression Example

Move 1 (Starting State, Random Selection)

- RL chooses a move randomly (exploration).
- Let's say it picks the **middle square (1,1)**.

The current estimated value of this state is **0.5**



Move 2 (Opponent's Turn)

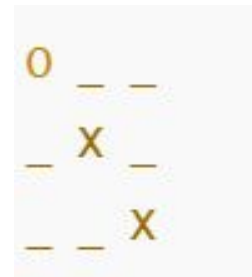
Opponent (O) moves to position **(0,0)**:

No learning happens yet because it's not our turn.



Move 3 (Greedy Move Selection)

- We check all possible moves and **pick the one with the highest value.**
- Assume our **table values suggest moving to (2,2)** is best.
- We move **X** to **(2,2)**.



Move 4 (Opponent Moves)

Opponent (O) moves to (0,1):

Again, no learning happens yet.

O	O	_
_	X	_
_	_	X

Move 5 (Greedy Move, Near Win)

We move **X** to **(1,2)**:

The estimated value of this state **increases** since it looks closer to a win.

O	O	_
_	X	X
_	_	X

Move 6 (Opponent Moves)

Opponent (O) moves to (0,2):

O	O	O
_	X	X
_	_	X

Step 3: Learning – Updating the Value Function

Since we lost, we update the values **backward** using the Temporal Difference (TD) Update Rule:

$$V(s) = V(s) + \alpha(V(s') - V(s))$$

where:

- $V(s)$ = value of the previous state
- $V(s')$ = value of the current state
- α = learning rate (e.g., 0.1)

The **learning rate** (denoted as α) is a parameter that controls **how much** the agent updates its knowledge after receiving new information.

Changes are based on a difference, $V(s) - V(s')$, between estimates at two different times

Role of Learning Rate (α):

- **High Learning Rate (e.g., $\alpha = 0.9$):**
 - Faster learning, but can be unstable.
 - The agent adapts quickly but may overreact to new information.
- **Low Learning Rate (e.g., $\alpha = 0.01$):**
 - Slower learning, but more stable.
 - The agent updates values slowly and does not forget past experiences quickly.

Updating State Values After Losing

Since we lost, our last move (before the loss) was **bad**, so we reduce its value:

1. **State Before Loss** (We moved to X at (1,2))
 - Old Value: 0.5
 - New Value: $0.5 + 0.1 (0 - 0.5) = 0.45$
2. **State Before That** (We moved X to (2,2))
 - Old Value: 0.5
 - New Value: $0.5 + 0.1 (0.45 - 0.5) = 0.495$

Now, when we play again, we will be **less likely** to make the same bad moves because their values have decreased.

Step 4: Playing More Games and Improving

- The more **games we play**, the more the value table gets **refined**.
- Over time, states that lead to **winning moves** will have higher values, and states that lead to **losing moves** will have lower values.
- Eventually, our RL agent will **learn the best strategy** by always making moves that **maximize the chance of winning**.

Example of Learned Value Table (Sample)

Learned value function estimates for Tic-Tac-Toe states using TD Learning.

Board State (Tuple Representation)	Value
(O, X, X, □, □, O, O, O, X, X)	0.061
(□, X, X, □, □, O, O, O, X, X)	0.193
(□, X, □, □, □, O, O, O, X, X)	0.593
(□, X, □, □, □, □, O, O, X, X)	0.437
(□, □, □, O, □, □, O, O, X, X)	0.477
(□, □, □, O, □, □, O, O, X, X)	0.477

□ represents an empty cell on the board.

Learned value function estimates for Tic-Tac-Toe states using TD Learning.

Expected Question

Illustrate the working of value-function reinforcement learning using Tic-Tac-Toe. How are values assigned, updated, and used for decision making? or Explain value-function reinforcement learning using Tic-Tac-Toe as an example. Illustrate how state values are initialized, updated, and used for decision making.

Why does value-function reinforcement learning learn faster than evolutionary methods in games like Tic-Tac-Toe?

What is the role of the learning rate (α) in updating the value table?

MENACE: Machine Educable Noughts and Crosses Engine.

link: <https://www.youtube.com/watch?v=R9c-neaxeU>

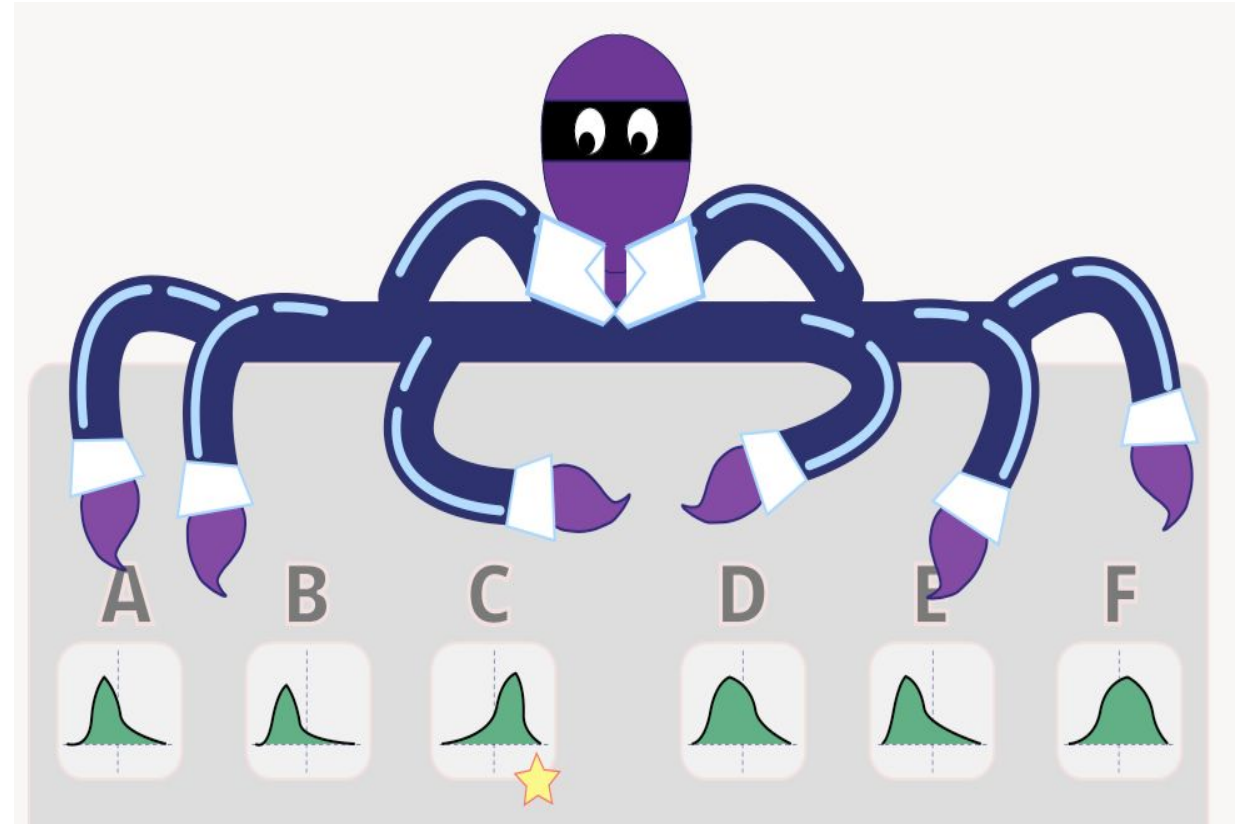
The Pile of Match Boxes that can learn at museum of science and industry manchester - a game with 304 Match boxes that learns an ideal policy.



Tabular Solution Methods

Multi-arm Bandits

- Reinforcement learning (RL) is different from other types of learning because it learns from feedback that evaluates actions rather than directly telling which action is correct. (**consequences and not instructions**) This means RL relies on trial and error to find good actions.
- There are two types of feedback:
 1. **Evaluative feedback** – tells how good an action was but not if it was the best or worst option. This is common in optimization methods like evolutionary algorithms.
 2. **Instructive feedback** – directly tells the correct action to take, regardless of what was chosen. This is used in supervised learning, such as neural networks.



These two types of feedback are usually separate but can sometimes be mixed.

To understand evaluative feedback better, we first study a simple version of reinforcement learning called the **n-armed bandit problem**. This problem helps introduce key learning methods before moving on to more complex RL scenarios where actions must be chosen in different situations.

An One Armed Bandit Problem



One Armed Bandit

A **one-armed bandit** is a nickname for a **slot machine**, the kind you see in casinos. It gets this name because:

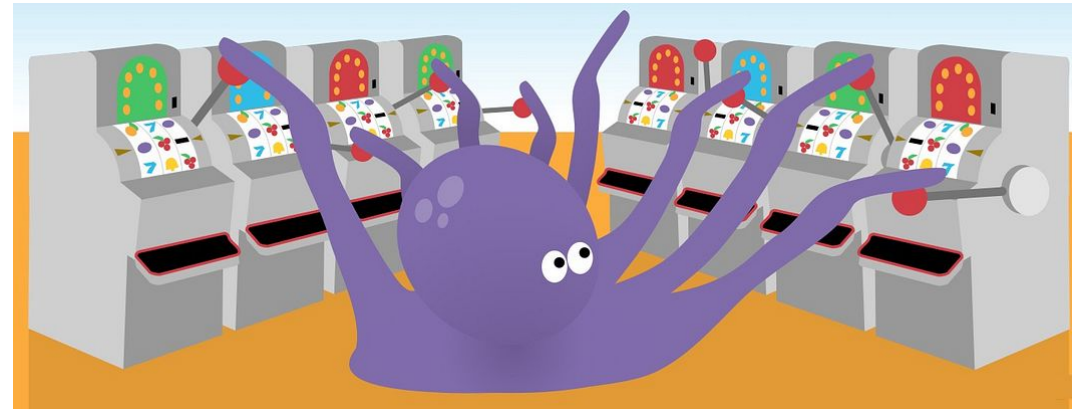
1. **"One-armed"** – Traditional slot machines have a **single lever** (or "arm") on the side that you pull to spin the reels. Modern machines often use buttons instead, but the name remains.
2. **"Bandit"** – The machine **takes your money** and rarely pays out big winnings, just like a thief or "bandit."

How Does a One-Armed Bandit Work?

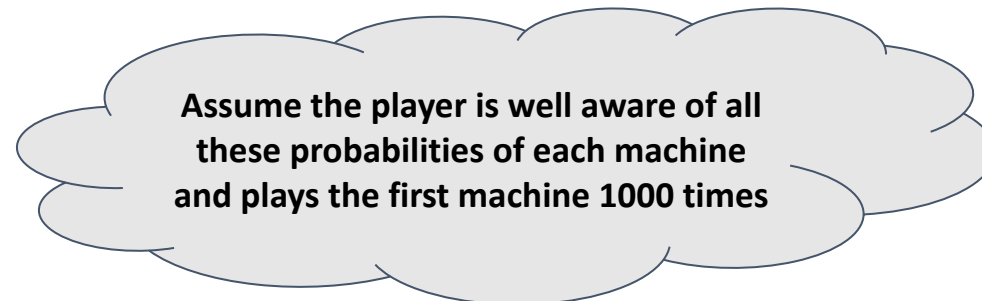
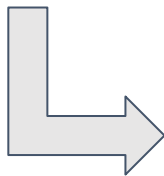
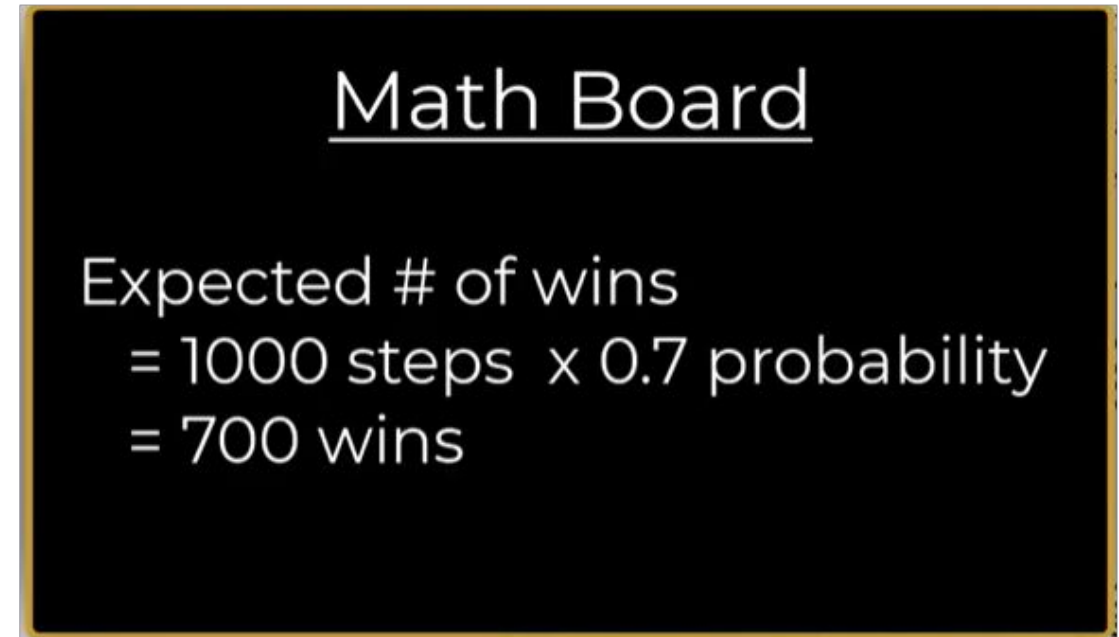
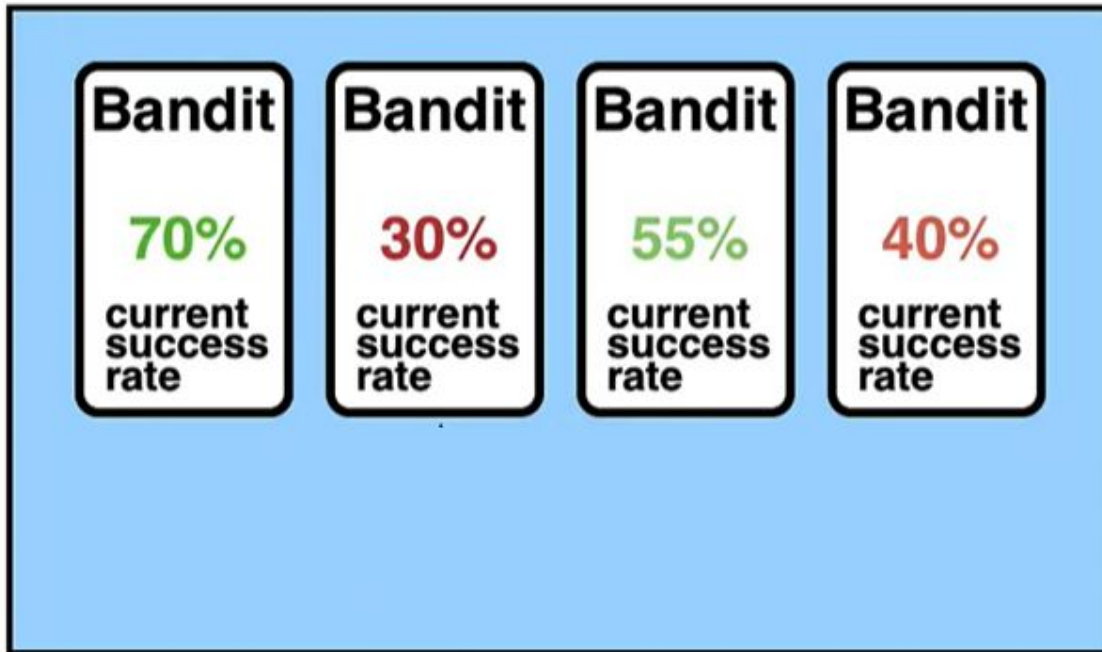
- You **insert money** (coins, bills, or a card with credits).
- You **pull the lever** (or press a button).
- The **reels spin** and then stop at random symbols.
- If the symbols **match a winning combination**, you receive a payout. Otherwise, you lose your bet.

n-Armed Bandit Problem

Imagine you walk into a casino filled with **slot machines** (also called "**one-armed bandits**" because of their lever). However, instead of just one machine, you have **n different slot machines** to choose from. Each machine gives out different amounts of money when you pull its lever, but you don't know in advance which one pays the most. Your goal is to **maximize your total winnings** over a fixed number of turns (let's say 1000).



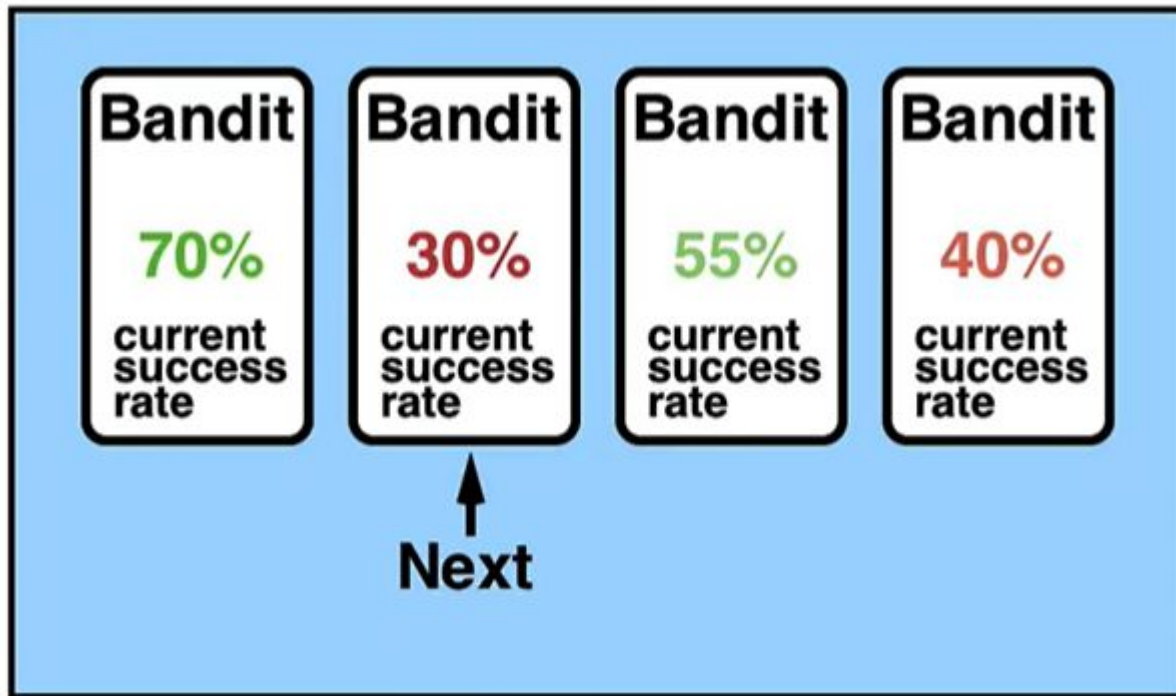
Now, think about how you should play to **win as much as possible**.



The non triviality of the multi armed bandit problem lies in the fact that we(the agent) **cannot access the true bandit probability distributions**- all learning is carried via trial & error and value estimation.

So we now have two method :

1. **Play the machine that has given you the highest rewards so far** (this is called **exploitation** – taking advantage of what you know).
2. **Try a different machine to see if it gives you a higher reward** (this is called **exploration** – gathering more information).



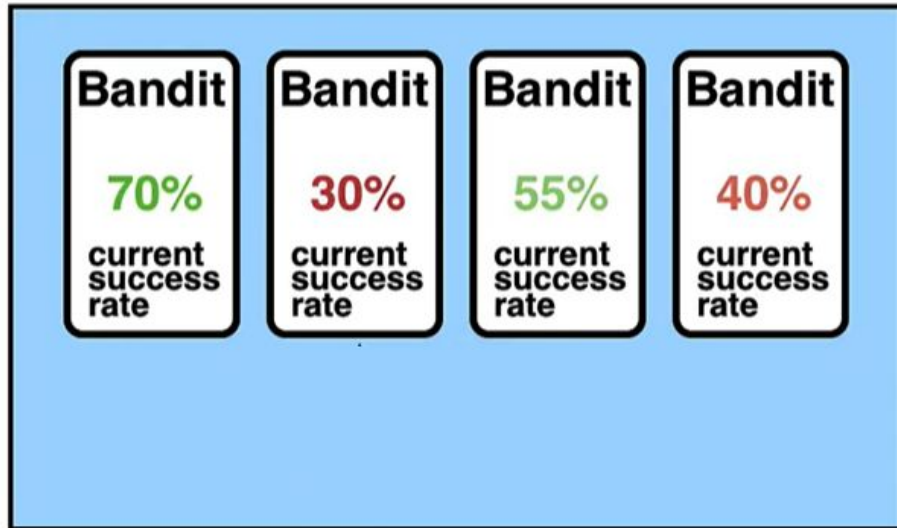
Math Board

Expected # of wins
= 1000 steps x 0.3 probability
= 300 wins

Agent tries only Machine 2 and thinks 30% success is the best.

Exploitation

Exploration: Tries all machines 250 times each.



Math Board

Expected # of wins
= 250 steps x (0.7 + 0.3 + 0.55 + 0.5)
= 250 * 1.95 wins
= 488 wins

This creates a **conflict** between exploration and exploitation:

- **If you always exploit(Greedy Selection)**, you might stick to a machine that seems good but isn't actually the best.
- **If you explore too much**, you might waste time on bad machines and miss out on higher rewards from the best machine.
- **The key challenge is finding the right balance between exploration and exploitation.**

Real Life Bandit Problems

The **multi-armed bandit (MAB) problem** models decision-making under uncertainty, making it useful in various real-world scenarios where we must balance **exploration (trying new things)** and **exploitation (using the best-known option)**.

- **Design of Experiments (Clinical Trials):** A doctor is testing multiple treatments for a disease but doesn't know which one works best. The doctor tries different treatments (exploration) while favoring the most effective one (exploitation).
- **Online Ad placement :** An advertiser has multiple ad versions but doesn't know which one gets the most clicks.
- **Website page persona:** A website has multiple page designs but doesn't know which one converts best (e.g., more sign-ups, purchases).
- **Games:** A game AI needs to choose the best strategy to win or adapt to a player's skill level.

Action-Value Methods- Greedy Method/ Sample Average Method

Lets explore simple balancing methods for the n-armed bandit problem and show that they work much better than methods that always exploit.

Action-value methods are simple techniques used in **reinforcement learning** to estimate the value of different actions and make decisions based on these estimates.

1. True Action Value ($q(a)$)

- This is the actual **expected reward** of an action **a**.
- If we could pull the same lever (take the same action) infinitely many times, the average reward would be $q(a)$.

2. Estimated Action Value ($Q_t(a)$)

- Since we don't know $q(a)$, we estimate it based on past rewards.
- At time step **t**, our estimate $Q_t(a)$ is the **average of all rewards** received when we previously chose action **a**.
- Formula:

$$Q_t(a) = \frac{R_1 + R_2 + \dots + R_{N_t(a)}}{N_t(a)}$$

- $N_t(a)$ = The number of times action **a** has been selected up to time **t**.
- $R_1, R_2, \dots, R_{N_t(a)}$ = The rewards received each time the action was chosen.

Action-value methods estimate the expected reward of each action by maintaining average reward values.

In the sample average method, these estimates are updated using the mean of observed rewards.

The greedy method selects the action with the highest estimated value to maximize immediate reward. However, purely greedy strategies may fail to explore better actions, making action-value methods a foundation for balancing exploration and exploitation in reinforcement learning.

A gambler is playing a **5-armed bandit machine**, where each arm provides a reward based on an unknown probability distribution. The gambler plays for **15 rounds**, receiving the following sequence of actions and rewards:

Time Step	Action Taken	Reward Received
1	A	6.0
2	B	7.5
3	C	3.8
4	D	6.9
5	E	2.5
6	A	5.5
7	B	5.8
8	A	4.8
9	C	4.5
10	D	7.4
11	B	6.2
12	C	4.1
13	D	6.8
14	B	6.1
15	A	5.2

Compute the estimated action values $Q(a)$ for each action after 15 rounds.
Which action has the highest estimated value, and should the gambler always select it moving forward?

Step 1: Compute the Estimated Action Values $Q(a)$

The estimated action value $Q_t(a)$ is calculated as:

$$Q_t(a) = \frac{\sum R}{N_t(a)}$$

where:

- $\sum R$ is the sum of all rewards received for that action.
- $N_t(a)$ is the number of times the action has been selected.

Let's compute the values step by step for each action.

Action A:

- Rewards received: 6.0, 5.5, 4.8, 5.2
- Number of times selected: 4

$$Q(A) = \frac{6.0 + 5.5 + 4.8 + 5.2}{4} = \frac{21.5}{4} = 5.38$$

Action B:

- Rewards received: 7.5, 5.8, 6.2, 6.1
- Number of times selected: 4

$$Q(B) = \frac{7.5 + 5.8 + 6.2 + 6.1}{4} = \frac{25.6}{4} = 6.40$$

Action C:

- **Rewards received: 3.8, 4.5, 4.1**
- **Number of times selected: 3**

$$Q(C) = \frac{3.8 + 4.5 + 4.1}{3} = \frac{12.4}{3} = 4.13$$

Action D:

- **Rewards received: 6.9, 7.4, 6.8**
- **Number of times selected: 3**

$$Q(D) = \frac{6.9 + 7.4 + 6.8}{3} = \frac{21.1}{3} = 7.03$$

Action E:

- **Rewards received: 2.5**
- **Number of times selected: 1**

$$Q(E) = \frac{2.5}{1} = 2.50$$

Step 2: Identify the Best Action

The final estimated values are:

Action	Final Estimated Value $Q(a)$
A	5.38
B	6.40
C	4.13
D	7.03
E	2.50

The action with the highest estimated value is D (7.03).

If we were following a greedy strategy, we would now always select D.

Limitation : If we always select the action with the highest estimated value (D), we risk missing better opportunities because:

- The estimates might still be inaccurate due to insufficient exploration.
- Actions B and A have relatively high values, and their true values might be higher than D.

Instead, the gambler should balance exploration and exploitation.

A digital marketing team is testing four different ad campaigns (A, B, C, D) for a product. Each ad generates revenue based on click-through rates (CTR), but the true effectiveness of each ad is unknown. The team follows a greedy method to maximize immediate revenue.

Given Data: Each ad has an unknown true average revenue per click, but the team only sees the revenue from each trial. They start testing with random ad selections and then switch to a greedy strategy (always picking the highest estimated ad).

They run 12 trials and record the following selections and observed revenues (in dollars):

Trial #	Ad Selected	Observed Revenue (\$)
1	A	5.0
2	B	6.0
3	C	4.5
4	D	7.5
5	B	6.2
6	D	7.2
7	A	5.3
8	B	5.8
9	D	7.0
10	D	7.4
11	D	7.1
12	D	7.3

- 1. Compute the estimated revenue for each ad after 12 trials using the greedy method.**
- 2. Identify which ad will be selected in the next round under the greedy approach.**
- 3. Explain the risk of using the greedy method in this case.**

Compute the Estimated Revenue for Each Ad

Ad	Estimated Revenue
A	5.15
B	6.00
C	4.50
D	7.25

Since Ad D has the highest estimated revenue (7.25), the greedy method will always select Ad D in the next round.

Risks of the Greedy Method

Although the greedy method maximizes short-term revenue, it has two major risks:

1. Lack of Exploration:

- Ad C was tested only once. It might actually be a high-revenue option, but because it was not tested enough, it gets ignored.
- The same applies to Ad A, which was tested only twice.

2. Potential for a Suboptimal Choice:

- The greedy method assumes Ad D is the best, but the estimates may still be inaccurate.
- A better strategy would be ϵ -greedy, where some fraction of selections (e.g., 10%) is used for exploring other ads.

Greedy Play : Suppose the reinforcement learning player was greedy, that is, it always played the move that brought it to the position that it rated the best. Would it learn to play better, or worse, than a nongreedy player? What problems might occur?

Lack	of	Exploration
Greedy player focuses only on immediate rewards. Prevents discovering new, potentially better strategies.		
Overfitting	to	Current Knowledge
Relies on inaccurate initial value estimates. (mis-estimates) Reinforces suboptimal actions based on misjudged rewards.		
Local		Optima
Sticks to strategies that seem optimal in the short term. May miss better long-term strategies that require suboptimal moves initially.		
Slow	Learning	and Convergence
Inefficient learning due to limited exploration. May take longer to converge to an optimal strategy.		
Non-greedy	Players	Perform Better
Non-greedy strategies (e.g., epsilon-greedy) balance exploration and exploitation. More likely to discover better overall strategies over time.		

Epsilon Greedy Method

The **ϵ -greedy algorithm** is a simple and effective **action-selection strategy** used in **reinforcement learning (RL)** and **multi-armed bandit problems**. It is designed to balance two conflicting objectives:

1. **Exploration** – Trying new actions to discover potentially better rewards.
2. **Exploitation** – Choosing the best-known action to maximize immediate rewards.

How the ϵ -Greedy Algorithm Works

First choose each arm once

Then at each step, the agent decides between:

- **Greedy action (exploitation):** With probability $(1-\epsilon)$, the agent selects the action with the highest estimated value.
- **Random action (exploration):** With probability ϵ , the agent selects a random action.

Mathematically, the action A_t at time step t is chosen as:

$$A_t = \begin{cases} \arg \max_i Q(i) & \text{with probability } 1 - \epsilon \text{ (exploitation)} \\ \text{random action} & \text{with probability } \epsilon \text{ (exploration)} \end{cases}$$

where:

- $Q(i)$ is the estimated value (expected reward) of action i .
- ϵ (epsilon) is a small number (e.g., 0.1) that controls the **exploration rate**.

What does ϵ mean?

ϵ = probability of exploration

ϵ value

Meaning

$\epsilon = 1$

Always choose randomly (full exploration)

$\epsilon = 0$

Always choose best arm (full exploitation)

ϵ between 0 and 1

Mix of both

Consider a 2 arm bandit

Step 1 : Choose each arm once - Arm 1

Step 2 : Choose each arm once - Arm 2

Step 3 **Assume $\epsilon = 1$ (Full exploration)**

Choose empirically best arm with probability 0%

Choose an arm uniformly at random with probability 100%

Step 4 Assume $\epsilon = 0.9$

Choose empirically best arm with probability 10%

Choose an arm uniformly at random with probability 90%

How to choose between best arm and random arm

Assume a random number $r = \text{np.random.rand()} = 0.48$

Choose empirically best arm if $r \geq \epsilon$

Choose an arm uniformly at random if $r < \epsilon$ i.e. $0.48 < 0.9$, so choose randomly

Step 4 Assume $\epsilon = 0.5$

Choose empirically best arm with probability 50%

Choose an arm uniformly at random with probability 50%

Assume we get a random number $r = \text{np.random.rand()} = 0.8$

Choose empirically best arm if $r \geq \epsilon$ i.e. $0.8 > 0.5$, so choose best arm

Choose an arm uniformly at random if $r < \epsilon$

Decision rule:

- Generate random number r between 0 and 1
- If $r < \epsilon \rightarrow$ Explore (choose random arm)
- If $r \geq \epsilon \rightarrow$ Exploit (choose best arm)



$$\begin{aligned} \text{Arm1} &: \frac{(0.9 + 0.7)}{2} = 0.8 \\ \text{Arm2} &: \frac{(0.5 + 0.3)}{2} = 0.4 \end{aligned}$$

Play Arm 1!

```

import numpy as np
true_means = [0.9, 0.3]

epsilon = 0.3
steps = 50
n_arms = 2
counts = np.zeros(n_arms)
values = np.zeros(n_arms)

np.random.seed(1)

# Play each arm once
for arm in range(n_arms):
    reward = np.random.normal(true_means[arm], 0.1)
    counts[arm] += 1
    values[arm] = reward

# Remaining steps
for step in range(n_arms, steps):
    if np.random.rand() < epsilon:
        action = np.random.randint(n_arms) # Explore
    else:
        action = np.argmax(values)         # Exploit

    reward = np.random.normal(true_means[action], 0.1)
    counts[action] += 1
    values[action] += (reward - values[action]) / counts[action]

print("Final Estimates:", values)
print("Counts:", counts)

```

<https://colab.research.google.com/drive/1SOUY35FWq610FHizThLHgc4xdYOr6odr?usp=sharing>

Write a Python program to implement the ϵ -greedy algorithm for a 2-armed bandit problem. Simulate at least 50 steps and display:

- Number of times each arm is selected
- Estimated value of each arm

Consider a 2-armed bandit problem solved using the ϵ -greedy method with $\epsilon = 0.3$.

Initially, each arm is played once and the following rewards are obtained:

Arm 1 $\rightarrow$ 0.9 Arm 2 $\rightarrow$ 0.5

For the next four steps, the random numbers generated are: 0.2, 0.7, 0.6, 0.1

The rewards received are:

	Step	Arm 1 Reward	Arm 2 Reward
	3	0.8	0.3
	4	0.7	0.4
	5	0.6	0.5
	6	0.85	0.6

Assume: If random number $r < \epsilon$, choose a random arm (exploration) Otherwise, choose the arm with the highest estimated value (exploitation)

Step 3 : choose **Arm 2** (for consistency).

Tasks:

For each step (3 to 6), determine whether the action is **exploration or exploitation**.

Identify **which arm is selected** at each step.

Update the estimated value of the selected arm using the **sample average method**.

Show the updated estimates after each step.

Find the **final estimated values** of both arms after Step 6.

Identify which arm is considered the **best** at the end.

$\epsilon = 0.3$

Rule:

If $r < 0.3 \rightarrow$ Exploration

Else $\rightarrow$ Exploitation (choose arm with highest estimated value)

Initial estimates: $Q_1=0.9, N_1=1$

$Q_2=0.5, N_2=1$

Step 3

$r = 0.2$

Since $0.2 < 0.3 \rightarrow$ Exploration

Choose Arm 2

Reward = 0.3 ☐

Update:

$$Q_2 = \frac{0.5 + 0.3}{2} = 0.4$$

$$N_2 = 2$$

$$Q_1 = 0.9, \quad Q_2 = 0.4$$

Now:

Step 4

$r = 0.7$

Since $0.7 \geq 0.3 \rightarrow$ Exploitation

Best arm = Arm 1 ($0.9 > 0.4$)

Reward = 0.7

Update: $Q_1 = \frac{0.9 + 0.7}{2} = 0.8$

$$N_1 = 2$$

$$Q_1 = 0.8, \quad Q_2 = 0.4$$

Step 5

$r = 0.6$

Since $0.6 \geq 0.3 \rightarrow$ Exploitation

Best arm = Arm 1

Reward = 0.6 ☐

Update: $Q_1 = \frac{0.9 + 0.7 + 0.6}{3} = \frac{2.2}{3} = 0.73$

$$N_1 = 3$$

$$Q_1 = 0.73, \quad Q_2 = 0.4$$

Step 6

$r = 0.1$

Since $0.1 < 0.3 \rightarrow$ Exploration

Choose Arm 2

Reward = 0.6

Update: $Q_2 = \frac{0.5 + 0.3 + 0.6}{3} = \frac{1.4}{3} = 0.47$

$$N_2 = 3$$

Final Results

Step	r	Action	Arm Chosen	Q1	Q2
1	–	Init	Arm 1	0.9	–
2	–	Init	Arm 2	0.9	0.5
3	0.2	Explore	Arm 2	0.9	0.4
4	0.7	Exploit	Arm 1	0.8	0.4
5	0.6	Exploit	Arm 1	0.73	0.4
6	0.1	Explore	Arm 2	0.73	0.47

Final Estimated Values

Q1=0.73 Q2=0.47

Best Arm at the end

Arm 1

Decay of ϵ :

In the equation:

$$\epsilon_t = \min \left\{ 1, \frac{Ck}{t\Delta_{\min}^2} \right\}$$

Explanation of Terms:

- ϵ_t : Exploration rate at time step t .
- C : A constant that controls the rate at which exploration decreases.
- k : The number of available actions (arms in a multi-armed bandit problem).
- t : The current time step (number of iterations or plays so far).
- $\Delta_{\min}$: The smallest gap between the expected rewards of the optimal action and the next best action.

At the beginning, the agent explores more to gather information.

Over time, exploration reduces, and the agent exploits the best-known option.

ϵ is adaptively reduced, leading to efficient learning.

This approach **balances exploration and exploitation**, ensuring the algorithm does not get stuck in suboptimal decisions.

Example

Suppose:

- $k = 2$
- $C = 1$
- $\Delta_{\min} = 0.5$

Then:

At $t = 1$: $\epsilon_1 = \min\left(1, \frac{2}{1 \times 0.25}\right) = 1$

At $t = 10$: $\epsilon_{10} = \frac{2}{10 \times 0.25} = 0.8$

At $t = 100$: $\epsilon_{100} = \frac{2}{100 \times 0.25} = 0.08$

Exploration reduces over time.

Decay of ϵ :

- The term $CK/t\Delta^2$ represents how ϵ decays over time. As t (the number of actions taken so far) increases, the term becomes smaller, meaning ϵ gets smaller as time goes on. This means the agent will gradually focus more on exploitation (i.e., choosing the best arm) over time, rather than exploring random arms.
- The factor CK scales the decay. C can be adjusted to control the rate of decay, and K is the number of arms, ensuring that more arms will influence the exploration rate.

The Role of Δ^2 :

- The term Δ^2 represents the squared difference between the optimal arm's reward (Q^*) and the estimated reward of a given arm ($Q(a)$).
- A larger Δ (larger gap between optimal and suboptimal arms) means the agent has more confidence in the optimal arm and is more likely to exploit it. This encourages the agent to focus more on exploitation as Δ grows.
- If Δ is small (the rewards of the arms are similar), the agent will explore more to better distinguish between the arms.

Optimal Arm: This is the arm (action) that provides the **highest expected reward** over time. It is the best arm among all available arms in terms of long-term return.

Suboptimal Arms: These are the arms that provide **lower expected rewards** compared to the optimal arm.

Delta (Δ) - The Gap Between Optimal and Suboptimal Arms: Δ represents the difference between the expected reward of the **optimal arm** (Q^*) and the expected reward of a **suboptimal arm** ($Q(a)$).

$$\Delta = Q^* - Q(a)$$

A **larger Δ** means that the optimal arm has a much higher expected reward than a suboptimal arm, giving the agent more confidence that choosing the optimal arm will consistently yield higher returns.

A **smaller Δ** means that the rewards of the optimal arm and the suboptimal arms are closer, which makes it harder for the agent to distinguish between them, requiring more exploration to learn which arm is truly the best.

Numerical Problem: ϵ -Decay in ϵ -Greedy

Consider a 2-armed bandit problem where the exploration rate decays over time

Given:

- Number of arms $k = 2$
- Constant $C = 1$
- Smallest reward gap $\Delta_{\min} = 0.5$
- True mean rewards:
 - Arm 1 = 0.8 (optimal)
 - Arm 2 = 0.5
- Current estimated values:
 - $Q_1 = 0.75$
 - $Q_2 = 0.55$
- Random numbers generated at each step:

Step (t)	Random number (r)
1	0.40
2	0.60
5	0.20
10	0.30

Tasks

1. Compute ϵ_t at each time step.
2. Determine whether the agent **explores or exploits**.
3. Identify which arm will be selected at each step.
4. Comment on how exploration changes over time.

the exploration rate decays over time according to:

$$\varepsilon_t = \min \left(1, \frac{Ck}{t\Delta_{\min}^2} \right)$$

Solution

First compute: $\Delta_{\min}^2 = (0.5)^2 = 0.25$

$$\varepsilon_t = \min \left(1, \frac{2}{0.25t} \right) = \min \left(1, \frac{8}{t} \right)$$

Step t = 1

$$\varepsilon_1 = \min(1, 8/1) = 1$$

$$r = 0.40$$

Since $r < 1 \rightarrow$ **Explore**

Arm chosen: Random (assume Arm 2)

Step t = 2

$$\varepsilon_2 = \min(1, 8/2) = 1$$

$$r = 0.60$$

Since $r < 1 \rightarrow$ **Explore**

Arm chosen: Random (assume Arm 1)

Step t = 5

$$\varepsilon_5 = \min(1, 8/5) = 1$$

$$r = 0.20$$

Since $r < 1 \rightarrow$ **Explore**

Arm chosen: Random

Step t = 10

$$\varepsilon_{10} = \min(1, 8/10) = 0.8$$

$$r = 0.30$$

Since $0.30 < 0.8 \rightarrow$ **Explore**

Arm chosen: Random

As time increases:

t	ε
1	1.0
2	1.0
If t = 20	$\varepsilon = 0.4$
If t = 50	$\varepsilon = 0.16$
If t = 100	$\varepsilon = 0.08$

Exploration gradually decreases.

Eventually:

- ε becomes small
- Agent mostly **exploits**
- Best arm = Arm 1 (since $Q_1 > Q_2$)

Consider a 3-armed bandit problem using the ϵ -greedy method with decaying ϵ .

Given

- Number of arms $k = 3$
- Constant $C = 0.5$
- Smallest gap $\Delta_{\min} = 0.4$
- Current estimated action values:
 - $Q_1 = 0.65$
 - $Q_2 = 0.80$
 - $Q_3 = 0.60$

Random numbers generated:

Step (t)	Random number (r)
1	0.30
3	0.70
5	0.40
15	0.20

Tasks

1. Compute ϵ_t for each time step.
2. Decide whether the action is **exploration or exploitation**.
3. Identify which arm will be selected.

Formula:

$$\varepsilon_t = \min \left(1, \frac{Ck}{t\Delta_{\min}^2} \right)$$

Values:

- $k = 3$
- $C = 0.5$
- $\Delta_{\min} = 0.4$
- Estimated values:
 - $Q_1 = 0.65$
 - $Q_2 = 0.80$ (highest)
 - $Q_3 = 0.60$

So during exploitation → choose Arm 2

Step 1: Simplify ε formula

$$\Delta_{\min}^2 = (0.4)^2 = 0.16$$

$$\varepsilon_t = \min \left(1, \frac{0.5 \times 3}{t \times 0.16} \right)$$

$$\varepsilon_t = \min \left(1, \frac{1.5}{0.16t} \right)$$

$$\varepsilon_t = \min \left(1, \frac{9.375}{t} \right)$$

Step t = 1

$$\varepsilon_1 = \min(1, 9.375/1) = 1$$

$$r = 0.30$$

Since $r < 1 \rightarrow$ Exploration

Arm selected: Random (assume Arm 3)

Step t = 3

$$\varepsilon_3 = \min(1, 9.375/3) = \min(1, 3.125) = 1$$

$$r = 0.70$$

Since $r < 1 \rightarrow$ Exploration

Arm selected: Random (assume Arm 1)

Step t = 5

$$\varepsilon_5 = \min(1, 9.375/5) = \min(1, 1.875) = 1$$

$$r = 0.40$$

Since $r < 1 \rightarrow$ Exploration

Arm selected: Random (assume Arm 2)

Step t = 15

$$\varepsilon_{15} = \min(1, 9.375/15) = \min(1, 0.625) = 0.625$$

$$r = 0.20$$

Since $0.20 < 0.625 \rightarrow$ Exploration

Arm selected: Random (assume Arm 3)

Final Answer Table

Step (t)	ϵ_t	r	Action	Arm Selected
1	1.00	0.30	Explore	Arm 3
3	1.00	0.70	Explore	Arm 1
5	1.00	0.40	Explore	Arm 2
15	0.625	0.20	Explore	Arm 3

Feature	Greedy Method	ϵ-Greedy Method
Exploration	Does not explore other actions once a best action is found	Explores other actions with probability ϵ
Exploitation	Always exploits the best-known action (highest estimated value)	Exploits the best-known action with probability $1-\epsilon$
Risk of Getting Stuck	High risk of getting stuck in a suboptimal solution (local maximum)	Lower risk as it occasionally explores better alternatives
Performance in Stationary Environments	Performs well when rewards are stable	Performs well when rewards are stable
Optimal for Non-Stationary Environments	Not ideal, since it does not adapt to changes	Better suited for non-stationary environments as it explores potential better actions

Questions

Explain the ϵ -greedy action selection method in the multi-armed bandit problem. How does the parameter ϵ control the exploration–exploitation trade-off?

Limitations of the Action-Value Method:

1. Memory Usage:

- The method requires maintaining a record of the rewards for each action, which could become memory-intensive if the number of actions is large, especially in real-world problems.

2. Computational Cost:

- In its straightforward form, the action-value method requires recalculating the average value each time an action is selected, which can be computationally expensive as the number of rewards increases for each action.

$$Q_t(a) = \frac{R_1 + R_2 + \dots + R_{N_t(a)}}{N_t(a)}$$

Incremental implementation

The solution to this is the use of **incremental updating** to compute the action-value estimate without having to store all the past rewards.

For some action, let Q_k denote the estimate for its k th reward, that is, the average of its first $k - 1$ rewards. Given this average and a k th reward for the action, R_k , then the average of all k rewards can be computed by

$$Q_{K+1} = \frac{R_1 + R_2 + R_3 + \dots + R_K}{K}$$

Add Summation

$$Q_{K+1} = \frac{1}{K} \sum_{i=1}^K R_i$$

$$Q_{K+1} = \frac{1}{K} \sum_{i=1}^K R_i$$

Separate R_K from Summation

$$Q_{K+1} = \frac{1}{K} \sum_{i=1}^{K-1} R_i + R_K \quad \text{--- (1)}$$

We know that

$$\begin{aligned} Q_K &= \frac{R_1 + R_2 + \dots + R_{K-1}}{K-1} \\ &= \frac{1}{K-1} \sum_{i=1}^{K-1} R_i \end{aligned}$$

Equate in (1)

$$Q_{K+1} = \frac{1}{K} \frac{(K-1)}{(K-1)} \sum_{i=1}^{K-1} R_i + R_K$$

$$= \frac{1}{K} (K-1) Q_K + R_K$$

$$= \frac{1}{K} [K Q_K - Q_K + R_K]$$

$$\boxed{Q_{K+1} = Q_K + \frac{1}{K} [R_K - Q_K]}$$

Final Incremental Update Formula

$$Q_{k+1} = Q_k + \frac{1}{k} (R_k - Q_k)$$

New estimate = Old estimate + Step size × Error

Where:

- Error = $(R_k - Q_k)$
- Step size = $\frac{1}{k}$

Advantages of incremental Approach

- The key benefit of this incremental formula is that it only requires storing the **current average** Q_k and the new reward R_k , avoiding the need to store and sum all previous rewards.
- The memory usage remains constant regardless of how many times the action is selected, and the computation per step is constant ($O(1)$).
- The action value is updated efficiently without needing to revisit all previous rewards.

Tracking a Nonstationary Problem

Consider example

Imagine you run an online store and track which products customers like the most. If you simply average all past sales, your recommendations may become outdated.

For example, if winter coats were popular months ago but now it's summer, those past sales shouldn't influence today's recommendations as much. Instead, using a constant step-size update, you can give more weight to recent purchases (like sunglasses or swimsuits) and adapt quickly to changing customer preferences.

This way, your store always suggests the most relevant products, keeping customers happy!

Tracking a Nonstationary Problem

In reinforcement learning, the **non-stationary n-armed bandit problem** is a variation of the classical **multi-armed bandit (MAB) problem**, where the reward distribution of each arm changes over time. This poses an additional challenge since optimal strategies must adapt to these changes instead of converging to a fixed optimal arm.

Changing Reward Distributions

- The expected reward of each arm may drift over time due to external factors.
- Changes could be abrupt (e.g., sudden shifts) or gradual (e.g., slow drift).
- Classical stationary bandit algorithms (like ϵ -greedy) assume fixed reward distributions.
- In non-stationary environments, older observations may become irrelevant, requiring algorithms to be more adaptive.

In such cases it makes sense to weight recent rewards more heavily than long-past ones. One of the most popular ways of doing this is to use a constant step-size parameter. Instead of treating all past rewards equally, we use a weighted average, where recent rewards are given more importance.

Consider the incremental Implementation equation for estimating the action value

$$Q_{k+1} = Q_k + \frac{1}{k} [R_k - Q_k]$$

The incremental update rule for updating an average Q_k of the $k - 1$ past rewards is modified to be

$$\begin{aligned} Q_{k+1} &= Q_k + \alpha [R_k - Q_k] \\ &= \alpha R_k + (1 - \alpha) Q_k \\ &= \alpha R_k + (1 - \alpha) [\alpha R_{k-1} + (1 - \alpha) Q_{k-1}] \\ &= \alpha R_k + (1 - \alpha) \alpha R_{k-1} + (1 - \alpha)^2 Q_{k-1} \\ &= \alpha R_k + (1 - \alpha) \alpha R_{k-1} + (1 - \alpha)^2 \alpha R_{k-2} + \\ &\quad \dots + (1 - \alpha)^{k-1} \alpha R_1 + (1 - \alpha)^k Q_1 \\ &= (1 - \alpha)^k Q_1 + \sum_{i=1}^k \alpha (1 - \alpha)^{k-i} R_i. \end{aligned}$$

How Are the Weights Assigned?

- The most recent reward R_k gets the highest weight.
- The older rewards R_i get **progressively lower** weights.
- The weights **decay exponentially** (shrink quickly as time passes).

The weight of a past reward R_i is:

$$\alpha(1 - \alpha)^{k-i}$$

where:

- α (alpha) = Step size (a small number like 0.1 or 0.01). It controls **how much importance the new reward gets**.
- $(1-\alpha)$ = A number **less than 1**, meaning that as time passes, the weight of past rewards gets **smaller and smaller**.
- $k-i$ = The difference between the current step k and the past step i , meaning **how long ago** the reward was received.

Let's say we are tracking the **average rating of a restaurant** over time, and we receive the following ratings:

Time Step (i)	Rating (R_i)	Weight Calculation
1 (oldest)	3.5	$\alpha(1 - \alpha)^{k-1}$
2	4.0	$\alpha(1 - \alpha)^{k-2}$
3	4.5	$\alpha(1 - \alpha)^{k-3}$
4 (latest)	5.0	$\alpha(1 - \alpha)^0 = \alpha$

- The most recent reward (5.0) gets the highest weight, which is just α .
- The second-most recent reward (4.5) gets a slightly lower weight: $\alpha(1 - \alpha)$.
- The older rewards (4.0 and 3.5) get even lower weights, as the exponent on $(1 - \alpha)$ increases.
- The oldest rewards eventually become almost irrelevant because their weight approaches zero.

This **exponential decay** ensures that we focus more on recent data while still keeping some memory of the past.

Optimal Initial Values in Reinforcement Learning

When training a reinforcement learning (RL) agent, **the initial values of action estimates ($Q(a)$) can significantly impact learning efficiency**. **Optimistic Initialization** is a technique where we **start with high initial values** to encourage exploration early on.

- Previous methods use 0 as the default initial value for the q value estimates
- This means they depend to some extent on the initial action-value estimates $Q_1(a)$
- You can also say they are biased by their initial estimates

Optimal Initial Values

Encourage Exploration

- Initial action values can also be used as a way to encourage exploration
- Suppose that we set the action values for all actions to +5 in 10-armed testbed problem instead of 0
 - As we discussed in part-1 that $q^*(a)$ true action values are selected from a normal distribution with mean 0 and variance 1.
 - Initial estimate of +5 is optimistic but it encourages exploration because whichever actions are initially selected the reward is less than starting estimates
 - This means the agent switches to other actions after being disappointed with the rewards it is receiving
 - This means that all actions are tried several times before the value estimates converge
 - The system does a fair amount of exploration even if greedy actions are selected all the time

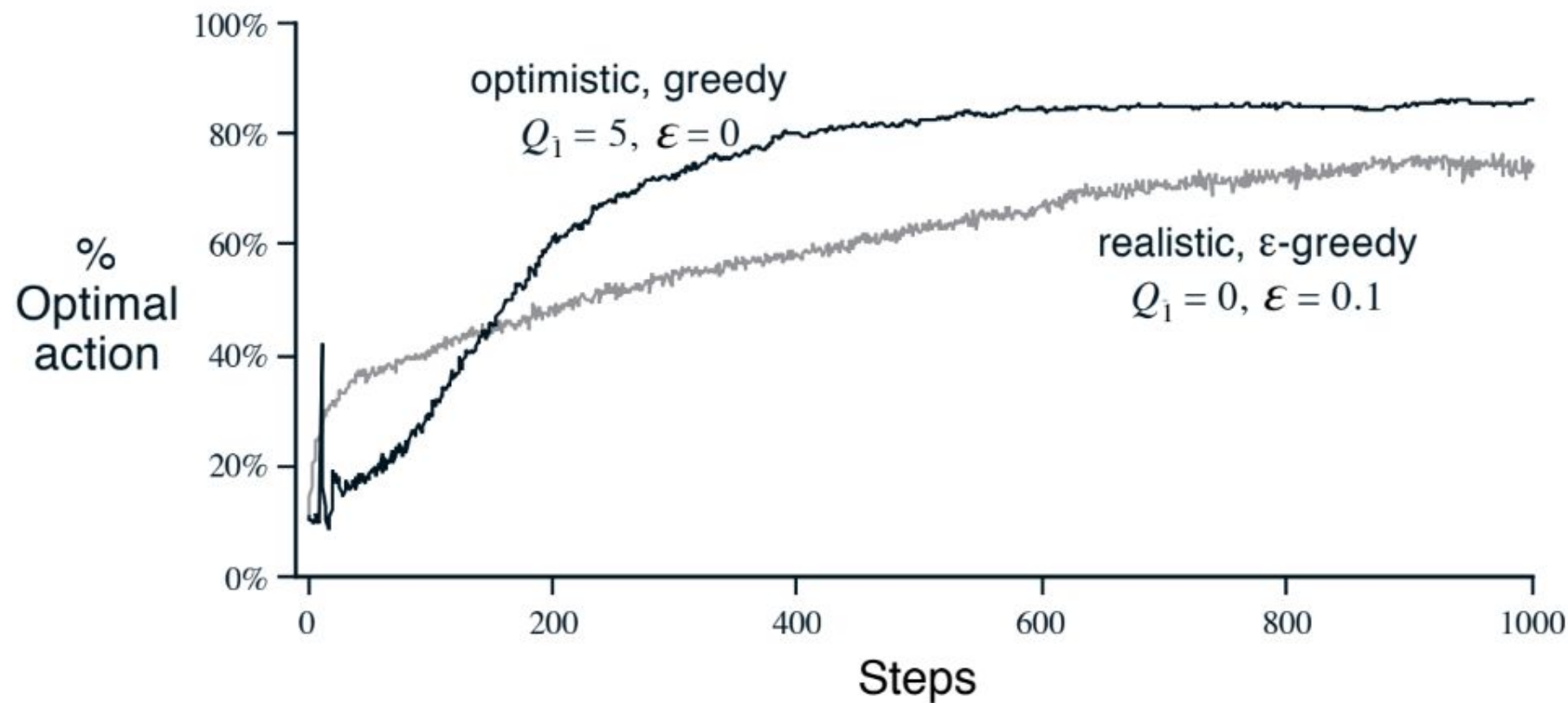


Figure 2.2: The effect of optimistic initial action-value estimates on the 10-armed testbed. Both methods used a constant step-size parameter, $\alpha = 0.1$.

Upper Confidence Bound

- There is always uncertainty about the accuracy in the action-value estimates
- Hence exploration is needed
- Epsilon-greedy action selection forces non-greedy action to be tried but it does so indiscriminately, with no preference for those that are nearly greedy or particularly uncertain
- It would be better to select non-greedy action according to their potential for actually being optimal
- This means taking into account
 - how close their estimates are to maximum value
 - and uncertainties in those estimates

One effective way of doing this is to select actions according to below equation

$$A_t \doteq \operatorname{argmax}_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

$$A_t \doteq \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

- $\ln t$ denotes the natural logarithm of t
- $N_t(a)$ denotes the number of times that action a has been selected prior to time t
- the number $c > 0$ controls the degree of exploration
- The square root term is a measure of uncertainty or variance in the estimate a 's action-value
- If $N_t(a) = 0$, then a is considered to be a maximizing action. (means a needs to be tried more)
- The quantity inside the square bracket is thus the upper bound on the possible true value of action a , with c determining the confidence level
- Each time a is selected the uncertainty is presumably reduced; $N_t(a)$ is incremented and, as it appears in the denominator of the uncertainty term, the term is decreased.
- However each time an action other than a is selected, t increases but the $N_t(a)$ does not, that increases the uncertainty
- The use of natural logarithm means that the increases get smaller over time but are unbounded
- All actions will eventually be selected but actions with lower value estimates, or that have already been selected frequently, will be selected with decreasing frequency over time

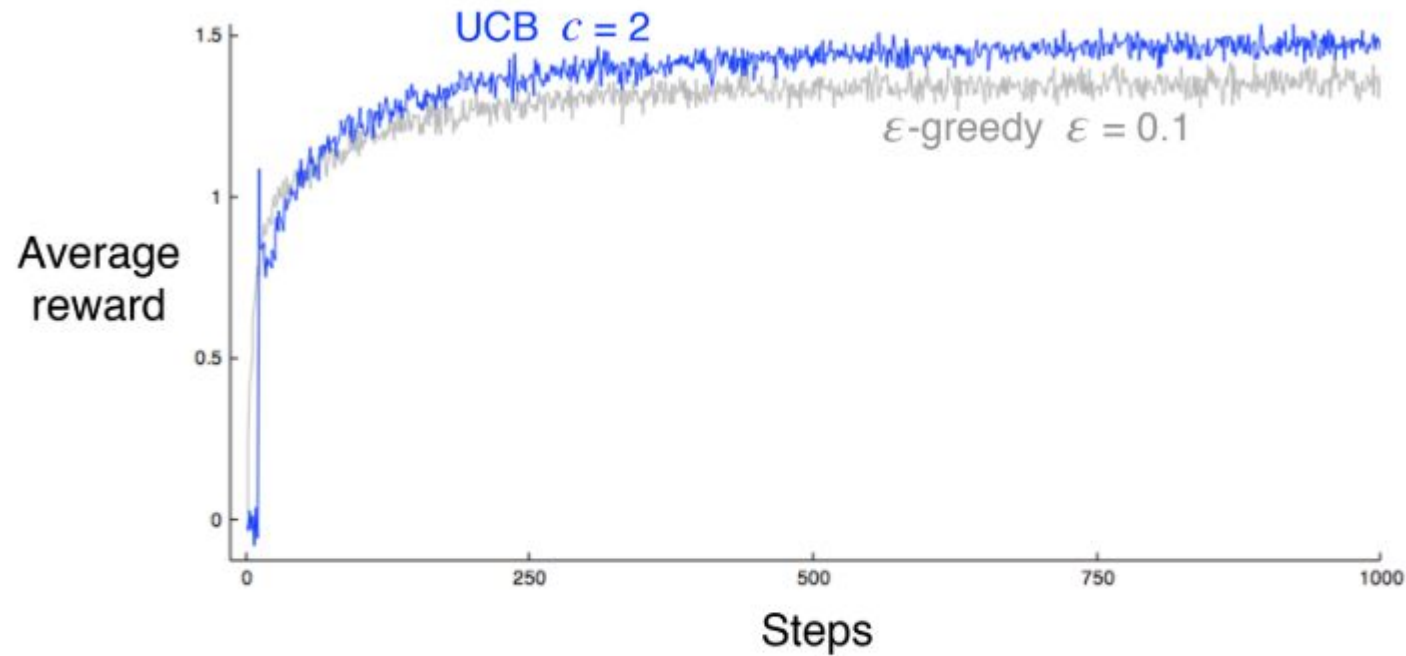


Figure 2.3: Average performance of UCB action selection on the 10-armed testbed. As shown, UCB generally performs better than ϵ -greedy action selection, except in the first n plays, when it selects randomly among the as-yet-unplayed actions.

So, in simple terms, UCB helps you explore uncertain options but gradually focuses on the best choice as you gather more data

Gradient Bandits

Gradient Bandit algorithms are a **reinforcement learning** approach where the agent doesn't estimate action values directly. Instead, it **learns preferences** for actions using a gradient-based update rule. These methods are useful when rewards are **non-stationary** or **unbounded**.

Here we consider learning a numerical preference $H_t(a)$ for each action a .

The larger the preference, the more often that action is taken, but the preference has no interpretation in terms of reward.

Instead of maintaining action-value estimates $Q(a)$, Gradient Bandits use **preferences** $H(a)$ for each action. These preferences determine a **softmax probability distribution (i.e., Gibbs or Boltzmann distribution)** for selecting actions. (The **softmax function** is a mathematical function that converts a set of real numbers into a probability distribution.)

Higher preferences correspond to higher selection probabilities.

1. **Compute Action Selection Probability** using the **softmax** function:

$$P_t(a) = \frac{e^{H_t(a)}}{\sum_b e^{H_t(b)}}$$

- Actions with **higher preferences** get **higher probabilities**.
- The agent explores naturally because probabilities are **spread out** over actions.

2. **Update Preferences** after receiving a reward:

$$H_{t+1}(A_t) = H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - P_t(A_t))$$

$$H_{t+1}(a) = H_t(a) - \alpha(R_t - \bar{R}_t)P_t(a) \quad \forall a \neq A_t$$

- R_t = reward received
- $\bar{R}_t$ = average reward (baseline to center updates)
- α = step size (learning rate)
- $P_t(A_t)$ = probability of choosing action A_t

The softmax function for a given action a at time t is defined as:

$$P_t(a) = \frac{e^{H_t(a)}}{\sum_b e^{H_t(b)}}$$

where:

- $H_t(a)$ is the **preference** or score assigned to action a .
- $e^{H_t(a)}$ (where $e \approx 2.718$) **exponentially scales** the preference.
- The denominator **normalizes** the values so that the probabilities sum to 1.
- The output $P_t(a)$ represents the probability of selecting action a .

Why Use Softmax?

- Ensures all probabilities sum to 1.
- Assigns **higher probability** to actions with higher preferences.
- Keeps a balance between **exploration** and **exploitation**.
- Unlike **ϵ -greedy**, it assigns **some probability** to all actions rather than selecting a random one.

Simple Example: Softmax in Action

Imagine you are choosing between **3 restaurants**, and you have assigned them the following **preference scores**:

Restaurant	Preference $H(a)$
 Pizza Spot	0.8
 Burger Place	1.2
 Sushi Bar	0.4

Burger Place has the highest preference, so it gets the highest probability (**47%**).

Pizza Spot has a lower preference but still has a significant chance (**32%**).

Sushi Bar has the lowest probability (**21%**), but it is **still chosen sometimes**.

Step 1: Compute Exponentials

We compute $e^{H(a)}$:

$$e^{1.2} \approx 3.32, \quad e^{0.8} \approx 2.23, \quad e^{0.4} \approx 1.49$$

Step 2: Compute Denominator

Sum up all exponentials:

$$3.32 + 2.23 + 1.49 = 7.04$$

Step 3: Compute Probabilities

$$P(Burger) = \frac{3.32}{7.04} \approx 0.47$$

$$P(Pizza) = \frac{2.23}{7.04} \approx 0.32$$

$$P(Sushi) = \frac{1.49}{7.04} \approx 0.21$$

Softmax transforms numerical scores into probabilities.

Updating Preferences in Softmax-Based Gradient Bandit Algorithms

In gradient bandit algorithms, we use preferences instead of estimated values (Q-values) to decide which action to take. These preferences are updated over time based on the rewards received, which helps the algorithm learn which actions are better.

The **preference** for an action $H(a)$ is updated using the formula:

$$H_{t+1}(a) = H_t(a) + \alpha \times (R_t - \bar{R}_t) \times (1 - P_t(a))$$

And for actions **not taken**:

$$H_{t+1}(b) = H_t(b) - \alpha \times (R_t - \bar{R}_t) \times P_t(b)$$

where:

- $H_t(a)$ = current preference for action a .
- α = step size (small value like 0.1).
- R_t = reward received after taking action a .
- $\bar{R}_t$ = average reward so far.
- $P_t(a)$ = probability of choosing action a (from softmax function).

1. If an action gets a higher reward than expected ($R_t > \bar{R}_t$):

- The preference for that action increases (more likely to be chosen).
- Other actions' preferences decrease slightly.

2. If an action gets a lower reward than expected ($R_t < \bar{R}_t$):

- The preference for that action decreases (less likely to be chosen).
- Other actions' preferences increase slightly.
- This helps maintain a probability distribution over actions and ensures exploration.

Step 1: Initial Setup

Let's assume we have 3 actions (A, B, and C) and their initial preferences:

Action	Initial Preference $H_t(a)$
A (🍷)	0.5
B (📺)	0.3
C (🎁)	0.2

Now, we apply the **Softmax function** to get the probabilities:

$$P_t(a) = \frac{e^{H_t(a)}}{\sum_b e^{H_t(b)}}$$

Step 2: Compute Softmax Probabilities

First, compute exponentials:

$$e^{0.5} = 1.65, \quad e^{0.3} = 1.35, \quad e^{0.2} = 1.22$$

Sum of exponentials:

$$1.65 + 1.35 + 1.22 = 4.22$$

Now, compute probabilities:

$$P(A) = \frac{1.65}{4.22} = 0.39$$

$$P(B) = \frac{1.35}{4.22} = 0.32$$

$$P(C) = \frac{1.22}{4.22} = 0.29$$

Step 3: Take Action and Receive Reward

- Suppose we choose Action A (☕ Café A) and receive a reward of $R_t = 9$.
- The average reward so far: $\bar{R}_t = 6$.
- Step-size parameter: $\alpha = 0.1$.

Since Action A got a higher-than-average reward, its preference should increase, and the other actions' preferences should slightly decrease.

Step 4: Update Preferences

Formula for the chosen action A:

$$H_{t+1}(A) = H_t(A) + \alpha(R_t - \bar{R}_t)(1 - P_t(A))$$

Substituting values:

$$H_{t+1}(A) = 0.5 + 0.1(9 - 6)(1 - 0.39)$$

$$H_{t+1}(A) = 0.5 + 0.1 \times 3 \times 0.61$$

$$H_{t+1}(A) = 0.5 + 0.183 = 0.683$$

✓ The preference for Action A has increased from 0.5 to 0.683!

Formula for actions NOT chosen (B and C):

$$H_{t+1}(b) = H_t(b) - \alpha(R_t - \bar{R}_t)P_t(b)$$

For Action B:

$$H_{t+1}(B) = 0.3 - 0.1(9 - 6)(0.32)$$

$$H_{t+1}(B) = 0.3 - 0.096 = 0.204$$

For Action C:

$$H_{t+1}(C) = 0.2 - 0.1(9 - 6)(0.29)$$

$$H_{t+1}(C) = 0.2 - 0.087 = 0.113$$

- ✓ **When an action gets a high reward, its preference increases, making it more likely to be chosen.**
- ✓ **Other actions' preferences decrease slightly to balance exploration.**
- ✓ **Over time, the best actions are chosen more often while still exploring others.**

A reinforcement learning agent is selecting between **three actions (A, B, and C)** in a gradient bandit setting. The initial preferences are:

$$H_0(A) = 0.4, \quad H_0(B) = 0.3, \quad H_0(C) = 0.2$$

Using the **Softmax function**, calculate the action probabilities.

If **Action A** is chosen and receives a **reward of 8**, with a **running average reward of 5**, update the preferences using **step size $\alpha = 0.1$** .

We first calculate the **exponentials** of the preferences:

$$e^{H_0(A)} = e^{0.4} \approx 1.4918$$

$$e^{H_0(B)} = e^{0.3} \approx 1.3499$$

$$e^{H_0(C)} = e^{0.2} \approx 1.2214$$

Now, we compute the **sum of the exponentials**:

$$\text{Sum} = e^{0.4} + e^{0.3} + e^{0.2} = 1.4918 + 1.3499 + 1.2214 \approx 4.0631$$

Now we calculate the **probabilities** for each action:

$$P(A) = \frac{e^{0.4}}{4.0631} = \frac{1.4918}{4.0631} \approx 0.367$$

$$P(B) = \frac{e^{0.3}}{4.0631} = \frac{1.3499}{4.0631} \approx 0.332$$

$$P(C) = \frac{e^{0.2}}{4.0631} = \frac{1.2214}{4.0631} \approx 0.300$$

Given:

- Step size $\alpha = 0.1$
- Selected action: Action A
- Reward for Action A $R = 8$
- Running average reward $\bar{R} = 5$

The preference update rule is:

$$H_{t+1}(a) = H_t(a) + \alpha(R_t - \bar{R})(1 - P(a))$$

For **Action A** (the chosen action):

$$H_1(A) = H_0(A) + 0.1 \times (8 - 5) \times (1 - P(A))$$

$$H_1(A) = 0.4 + 0.1 \times 3 \times (1 - 0.367)$$

$$H_1(A) = 0.4 + 0.1 \times 3 \times 0.633$$

$$H_1(A) = 0.4 + 0.190 \approx 0.590$$

For **non-chosen actions** (Actions B and C), we update their preferences as follows:

For **Action B**:

$$H_1(B) = H_0(B) - 0.1 \times (8 - 5) \times P(B)$$

$$H_1(B) = 0.3 - 0.1 \times 3 \times 0.332$$

$$H_1(B) = 0.3 - 0.099 \approx 0.201$$

For **Action C**:

$$H_1(C) = H_0(C) - 0.1 \times (8 - 5) \times P(C)$$

$$H_1(C) = 0.2 - 0.1 \times 3 \times 0.300$$

$$H_1(C) = 0.2 - 0.090 \approx 0.110$$

Assignment

An agent follows a gradient bandit algorithm with the following initial preferences:

$$H_0(A) = 0.6, \quad H_0(B) = 0.2, \quad H_0(C) = 0.1$$

- Calculate the initial action probabilities using the **Softmax function**.
- Suppose **Action B is selected** and receives a **reward of 6**, while the **average reward is 4**.
- Update the preferences using $\alpha = 0.2$.
- Then, suppose **Action A is selected next** and receives a **reward of 9**, while the new **average reward is 5.5**.
- Update the preferences again.

